

REFERENCIA COMPLETA – MÉTODOS DE RESULTSET

Método	Tipo devuelto	Uso
next()	boolean	Avanza al siguiente registro. <code>false</code> = no hay más.
getString("col")	String	Lee columna VARCHAR, TEXT, CHAR
getInt("col")	int	Lee columna INT, SMALLINT
getDouble("col")	double	Lee columna DOUBLE, FLOAT
getFloat("col")	float	Lee columna FLOAT
getLong("col")	long	Lee columna BIGINT
getBoolean("col")	boolean	Lee columna BOOLEAN / TINYINT(1)
getDate("col")	java.sql.Date	Lee columna DATE
getTimestamp("col")	Timestamp	Lee columna DATETIME / TIMESTAMP
getObject("col")	Object	Lee cualquier tipo (requiere cast)
wasNull()	boolean	¿El último valor leído era NULL en BD?
getRow()	int	Número de fila actual (1-based)
close()	void	Cierra el ResultSet y libera recursos

ACCESO POR NOMBRE VS POR ÍNDICE

```
// Por nombre de columna (recomendado)
String nombre = rs.getString("nombre");
int id = rs.getInt("id");

// Por índice de columna (no recomendado)
// El índice empieza en 1, no en 0
String nombre = rs.getString(2); // 2ª columna del SELECT
```

El acceso por índice depende del orden del `SELECT`. Si cambias la consulta, el índice deja de ser correcto. Usa siempre el nombre de columna.

✗ ERRORES COMUNES

No llamar `rs.next()` antes de leer → `SQLException: Before start of result set`

Leer datos fuera del bucle `while` → cursor ya está al final, datos inaccesibles

Nombre de columna incorrecto → `SQLException: Column not found`

Usar `getInt()` en columna VARCHAR → `NumberFormatException`

El `ResultSet` solo se puede recorrer hacia adelante por defecto. Para navegación bidireccional necesitas `TYPE_SCROLL_INSENSITIVE`.

📁 PATRÓN: RESULTSET → ARRAYLIST DE OBJETOS (POJO)

Patrón más usado en proyectos reales: convertir cada fila del `ResultSet` en un objeto Java y acumularlos en una lista.

CLASE POJO (PLAIN OLD JAVA OBJECT)

```
public class Usuario {
    private int id;
    private String nombre;

    public Usuario(int id, String nombre) {
        this.id = id; this.nombre = nombre;
    }

    public int getId() { return id; }
    public String getNombre(){ return nombre; }
    @Override
    public String toString() {
        return id + " - " + nombre;
    }
}
```

MÉTODO QUE DEVUELVE LIST<USUARIO>

```
public static List<Usuario> obtenerUsuarios(
    Connection conn) throws SQLException {

    List<Usuario> lista = new ArrayList<>();
    String sql = "SELECT * FROM usuarios";

    try (PreparedStatement ps =
        conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            lista.add(new Usuario(
                rs.getInt("id"),
                rs.getString("nombre")));
        }
    }
    return lista;
}
```

📊 CÁLCULOS SOBRE RESULTSET

SUMA, MEDIA Y MÁXIMO RECORRIENDO EL RESULTSET

```
int suma = 0, contador = 0;
double maximo = Double.MIN_VALUE;

while (rs.next()) {
    double precio = rs.getDouble("precio");
    suma += precio;
    contador++;
    if (precio > maximo) maximo = precio;
}

double media = (contador > 0) ? ((double)suma/contador : 0;
System.out.println("Suma: "+suma+" | Media: "+media+" | Max: "+maximo);
```

Alternativa SQL más eficiente: `SELECT SUM(precio), AVG(precio), MAX(precio) FROM productos` — deja el cálculo al motor de BD.

ResultSet y Lectura de Datos

BLOQUE 4 · CHULETARIO · TRÍPTICO A4

FUNCIONAMIENTO DEL CURSOR RESULTSET

antes de start

- inicio -

cursor

rs.next() → fila 1

1 | Juan | Madrid

+ aquí

rs.next() → fila 2

2 | Ana | Sevilla

rs.next() → fila 3

3 | Pedro | Valencia

rs.next() → false

- fin -

TABLA SQL ↔ TIPOS JAVA

SQL	Método	Java
INT	getInt()	int
VARCHAR	getString()	String
DOUBLE / FLOAT	getDouble()	double
DATE	getDate()	java.sql.Date
BOOLEAN	getBoolean()	boolean

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq.** — Concepto de `ResultSet`, cursor, recorrido básico y completo
- Interior cent.** — Lectura de columnas, una fila, sin resultados, formatos
- Interior der.** — `ArrayList` de objetos, filtrado, tabla en consola
- Solapa** — Patrón POJO completo, cálculos sobre `ResultSet`
- Contraportada** — Tabla de todos los métodos, nombre vs índice, errores

🔍 ¿QUÉ ES RESULTSET?

`ResultSet` es el objeto que contiene las filas devueltas por un `SELECT`. Funciona como un **cursor** que apunta a las filas una a una. Al crearse, está posicionado *antes del primer registro*.

CARACTERÍSTICAS CLAVE

- Recorre solo hacia adelante por defecto
- Hay que llamar a `next()` antes de leer cualquier dato
- Cada llamada a `next()` avanza una fila
- Devuelve `false` cuando no hay más filas
- Los datos deben leerse dentro del bucle

🔄 RECORRER TODOS LOS RESULTADOS

```
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    // se ejecuta una vez por cada fila
    System.out.println("Fila encontrada");
}

// Cuando next() devuelve false → bucle termina
```

📄 EJEMPLO COMPLETO – SELECT CON PREPAREDSTATEMENT

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/prueba";

        try {
            Connection conn =
                DriverManager.getConnection(url, "root", "1234");
            PreparedStatement ps =
                conn.prepareStatement("SELECT * FROM usuarios");
            ResultSet rs = ps.executeQuery()
        } {
            while (rs.next()) {
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                System.out.println(id + " - " + nombre);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Código	Qué hace
<code>ps.executeQuery()</code>	Ejecuta el SELECT y devuelve el ResultSet
<code>rs.next()</code>	Avanza al siguiente registro. Condición del while.
<code>rs.getInt("id")</code>	Lee la columna "id" de la fila actual como int
<code>rs.getString("nombre")</code>	Lee "nombre" de la fila actual como String

✳️ VARIANTES DE LECTURA

LEER SOLO UNA COLUMNA

```
while (rs.next()) {
    System.out.println(rs.getString("nombre"));
}
```

LEER VARIAS COLUMNAS CON FORMATO

```
while (rs.next()) {
    int id = rs.getInt("id");
    String nombre = rs.getString("nombre");
    System.out.println("ID: " + id + " - Nombre: " + nombre);
}
```

LEER EXACTAMENTE UNA FILA (IF EN VEZ DE WHILE)

```
if (rs.next()) {
    // solo entra si hay al menos una fila
    System.out.println(rs.getString("nombre"));
} else {
    System.out.println("No se encontró el registro");
}
```

MANEJAR EL CASO SIN RESULTADOS

```
ResultSet rs = ps.executeQuery();
if (!rs.next()) {
    System.out.println("La tabla está vacía");
} else {
    do {
        System.out.println(rs.getString("nombre"));
    } while (rs.next());
}
```

MOSTRAR RESULTADOS EN FORMATO TABLA EN CONSOLA

```
System.out.printf("%-5s %-15s %-10s\n",
    "ID", "NOMBRE", "PRECIO");
System.out.println("-".repeat(32));
while (rs.next()) {
    System.out.printf("%-5d %-15s %-10.2f\n",
        rs.getInt("id"),
        rs.getString("nombre"),
        rs.getDouble("precio"));
}
```

CONTAR FILAS MANUALMENTE

```
int contador = 0;
while (rs.next()) contador++;
System.out.println("Total registros: " + contador);

// Alternativa SQL más eficiente:
// SELECT COUNT(*) AS total FROM tabla
```

Cuándo usar if vs while:
`if (rs.next())` → esperas exactamente 1 fila (login, buscar por ID)
`while (rs.next())` → esperas 0 o más filas (listados)

📦 RESULTSET → ARRAYLIST

```
List<String> nombres = new ArrayList<>();

while (rs.next()) {
    nombres.add(rs.getString("nombre"));
}

// Ahora puedes trabajar con la lista
// aunque la conexión esté cerrada
for (String n : nombres)
    System.out.println(n);
```

📦 RESULTSET → ARRAYLIST DE OBJETOS

```
List<Usuario> lista = new ArrayList<>();

while (rs.next()) {
    Usuario u = new Usuario(
        rs.getInt("id"),
        rs.getString("nombre"));
    lista.add(u);
}

// Usar la lista fuera del try-with-resources
lista.forEach(System.out::println);
```

🔍 FILTRAR EN JAVA (POST-RESULTSET)

```
while (rs.next()) {
    double precio = rs.getDouble("precio");
    // Filtro en Java, no en SQL
    if (precio > 50.0) {
        System.out.println(
            rs.getString("nombre") + " → " + precio);
    }
}
```

Filtrar en SQL con `WHERE precio > ?` es mucho más eficiente. Filtra en Java solo si la condición es compleja o ya tienes los datos cargados.

📄 TABLA RESUMEN DEL BLOQUE

Situación	Patrón
Listar todos	<code>while(rs.next())</code>
Buscar uno por ID	<code>if(rs.next())</code>
Comprobar si existe	<code>boolean existe = rs.next()</code>
Tabla vacía	<code>if(!rs.next()) → mensaje</code>
Guardar en lista	<code>lista.add(new Objeto(rs.getX...))</code>
Calcular total SQL	<code>SELECT COUNT(*) / SUM() / AVG()</code>

Regla de oro: llama siempre a `rs.next()` antes de leer · lee dentro del bucle · guarda en `ArrayList` si necesitas los datos fuera del bloque try · usa nombres de columna, no índices.